CG2271 Real Time Operating Systems
Assignment 4 & Mini Project – Putting it all together
Version 2.0

1. Introduction

Congratulations on making it to the final assignment! This assignment is an open-ended project, to use all the skills and concepts you have learnt throughout the semester. As announced in Lecture 0, this mini project will constitute **15%** of your grade for this course.

In this course we have looked at how to program bare metal on the MCXC444, and how to combine this bare-metal programming with FreeRTOS. The objective of this assignment is to give you a feel for developing a FreeRTOS-based system from scratch.

You will build an IoT system that incorporates sensors, actuators, communication and network access (via an ESP32 module), coordinating everything using FreeRTOS.

2. Submission Instructions

There are three submissions required:

1. Interim report (5 marks): Due **Week 10 Friday (23 October 2026), 2359H**
   Please submit your interim code together with the interim report
2. Project demo (35 marks): Due **Week 13 Lab Session**
3. Final report (10 marks): Due **Week 13 Friday (13 November 2026), 2359H**
   Please submit your final code together with the final report (under Final Demo section of Canvas).

The demo of your project will be held in your final lab in **Week 13**. Details of each component of your submission are in the following pages.

3. Provided Logistics

Each team will be provided with the following items:

- 1 x FRDM Development Board for MCXC444
- 1 x ESP32 Development Module + USB Cable
- 1 x Sensor Kit
- 2 x Mini Breadboard
- Assorted Jumper Wires

Apart from these materials provided, we have a limited supply of actuators/sensors available (e.g., DC motor, Dot Bar Display, more sensor kits). Please reach out to your TA to obtain these items.

You are welcome to source additional materials, though this is not required. (i.e., the grading will not be based on how many extra materials you purchase). Please note that there will be no reimbursement for any materials purchased.

Project Description

Project Requirements

This project requires you to create a **coherent product** using the ESP32 and MCXC444, using all the concepts you were taught during the lab sessions throughout the semester.

The requirements are as follows:

For 4/5 member groups: You should use 4 sensors and 3 actuators. For 3 member groups: You should use 3 sensors and 2 actuators. At least 1 sensor should be connected to the MCXC444. You may use up to 1 sensor and 1 actuator that is on board the MCXC444.
At least 1 sensor should be polled, and at least 1 sensor should be interrupt-driven.
You must have at least 1 ISR and 3 Tasks.
Ensure good ISR and task design.
You must use at least 1 Mutex or Semaphore and 1 queue.
You must enable time-slicing and pre-emption, removing any unnecessary components.
Data must be exchanged between the MCXC444 and ESP32. Note that this should be a **bi-directional** data exchange (i.e. Both devices should send AND receive data.)
The ESP32 must connect to the internet to exchange data with cloud-based services. This includes performing database operations, executing REST API calls, and utilising cloud inferencing (e.g., sending data to the OpenAI API).
The project should be coherent (i.e. the sensors and actuators should work to serve a single purpose. It should not be the case where the LED is driven solely by a timer, and the sensor data is not used for anything).
A good project should define a good use case, and all the sensors and actuators should work together to meet that objective.

Interim Report (5 marks)

You are required to write a short, **2-page** interim report. This report is not meant to be a full detailed report and serves to inform the teaching team on your project architecture, purpose and components used. This is to ensure that the project is sufficiently complex, and that you are on track to completing the project by Week 13.

The formatting details are as follows. Please use the same formatting for the final report as well.

- Standard font, font size 12.
- 1.15 spacing
- Justified alignment

This report is graded on the completeness of your plan for the project, and should include the following:

Problem statement of your project. This should be a simple 1 to 2 sentences on what your project aims to do.
Sensors and actuators used.
General architecture of the project (a description of which sensors/actuators are connected to the ESP32/MCXC444 and the purpose they serve). You may include hardware and software architecture (e.g., elaboration on how you intend to fulfill the requirement of using internet to exchange data with cloud-based services)
Current progress of the project.

This report will be due on **Friday, 23 October 2026, 2359**. Submit the report in PDF format, titled B0X_YY_Interim.pdf, where X is your assigned lab slot, and YY is your group number (Eg: B01_10_Interim.pdf). **Together with your interim report, please submit all source code in a zip file**, titled B0X_YY_Interim_Code.zip (Eg: B01_10_ Interim _Code.zip). Submit both the PDF and zip file under Interim Report on Canvas. **DO NOT** zip the report.

Teams will receive feedback about their interim report a week after the submission.

Final Report (10 marks)

You are required to write a **5-to-10-page** report on your project. This report explains more about the technical aspects of your project that may not have been covered during the demo.

As this final report is a longer version of your Interim Report, you should be able to mostly continue from the Interim Report.

The report is graded on the following criteria:

Introduction to your project. This should include the problem statement and feature list.
Sensors and actuators used. This should include data handling (what data you receive from the sensors, and how the actuators use it) and libraries used.
Architecture of the project. This should include: - Hardware Architecture → Circuit diagram or pin table → Sensor/actuator programming - Software Architecture → FSM (if necessary) → Task Structure → Coordination & Synchronization → Resource Protection - Communication protocol(s) between ESP32 and MCXC444 - Application to an online function
Online function used → Description of function used → Data sent and received by ESP32
Any special features
Challenges faced & Solutions

This report will be due on **Friday, 13 November 2026, 2359**. Submit the report in PDF format, titled B0X_YY_Final.pdf, where X is your assigned lab slot, and YY is your group number (Eg: B01_10_Final.pdf).

Project Demo (35 marks)

You are required to demonstrate all notable features of your project, in a 5–7-minute demonstration. During the lab slot of week 13, you will showcase your projects at your tables, and the TAs will go around to grade your projects. You are required to be present during your demo, but when you are not assessed you can walk around the lab to see each other's projects.

The demo should highlight not just the key features that we can observe (e.g. actuator activation) but also include certain useful aspects of the code that enhance the project's usability.

You should also show the TAs your use of RTOS within the project, with reference to your code to identify the various tasks, priorities and ISR design.

Together with your final report, please submit all source code in a zip file, titled B0X_YY_Code.zip (Eg: B01_10_Code.zip). This is to allow the teaching team to verify the use of RTOS concepts, should we need a second look after the demo. Please do so under the submission folder: Project Demo. Please note that we might be recording your demo for grading purposes. More information about the project demo will be shared closer to the day.

The rubrics for the project are as follows:

Sufficient sensors/actuators used	4/5-member group: 4 sensors, 3 actuators 3-member group: 3 sensors, 2 actuators Up to 2 marks will be given for good use of components. This refers to being able to make strong engineering justification on why certain sensor is used on certain board, why it is polled/driving interrupts, explaining the priority levels etc.	5 marks
Polling & Interrupt used	At least 1 sensor is polled At least 1 sensor drives interrupt	2 marks
ISR Design	Does not block any tasks/use blocking functions for extended periods of time Does not wait for inputs (eg UART) Correct use of synchronization tools	3 marks
Task Design	At least 3 tasks Yields to other tasks No obvious bugs (that causes noticeable errors)	4 marks
Synchronization tools	At least 1 Semaphore/Mutex used At least 1 Queue used	2 marks
RTOS enabled	Time-slicing is enabled Pre-emption is enabled	2 marks

ESP32 use	ESP32 is used meaningfully (i.e. not just to receive data and connect to the Internet) Uses more than just `digitalRead`, `digitalWrite`, `analogRead` and `analogWrite`. Libraries are used beyond the template provided.	7 marks
Communications Protocol(s)	Good use of 2-way communications. Up to 2 marks will be given for going beyond UART taught in lab 3, which includes, but not limited to: • Packetization • Inter-board communication control flows • Different serial protocols or encryption	5 marks
Online function(s)	The online function should be non-trivial (i.e. more than just a ping to show that it's connected). Up to 2 marks will be given for advanced internet connectivity, which includes, but not limited to: • Querying Generative AI APIs for inferencing • Reading from or writing to online databases • Exchanging data with a web application • MQTT	5 marks

Annex A: Connecting to the Internet

The ESP-32 has a built-in Wi-Fi chip, which allows you to connect to the Internet. This can be done with a built-in library in Arduino IDE, Wifi.h.

In the template Arduino Sketch provided titled `as4_sample.ino`, we have done a simple Wi-Fi example. There are a few parameters that you are required to change, namely the Wi-Fi SSID and Password, which is defined in constants at the top. You are required to use your own personal hotspot to connect to the Internet, as you may not be able to connect to NUS_STU directly.

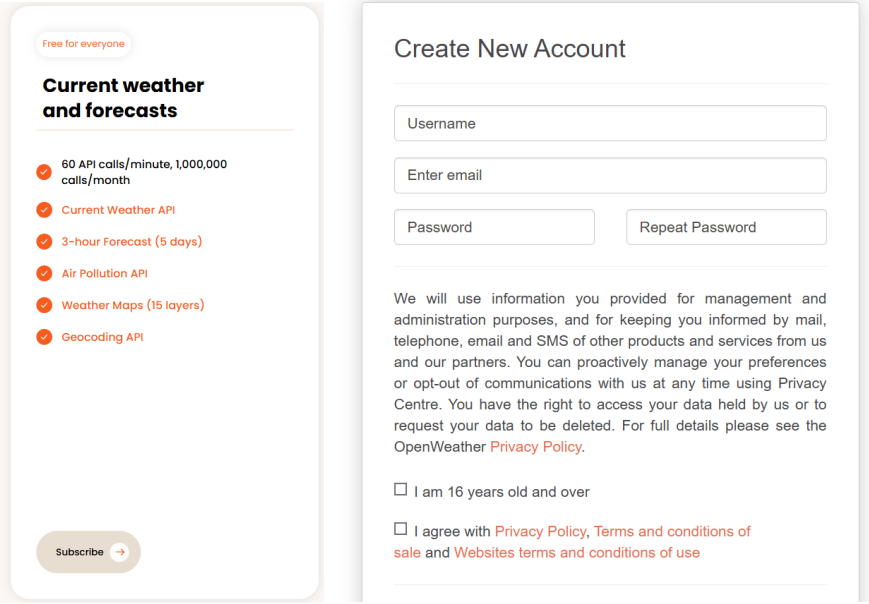
As connecting to the Internet is often done in setup (as is the case here), the main loop of the sketch is intentionally left blank.

The next step would be to connect to NUS_STU, which we have left for you to explore. A hint for you is that NUS_STU is a WPA2 Enterprise system, and requires a SSID, Username and Password. This is not necessary, and you should be able to finish the project solely on your own personal hotspot.

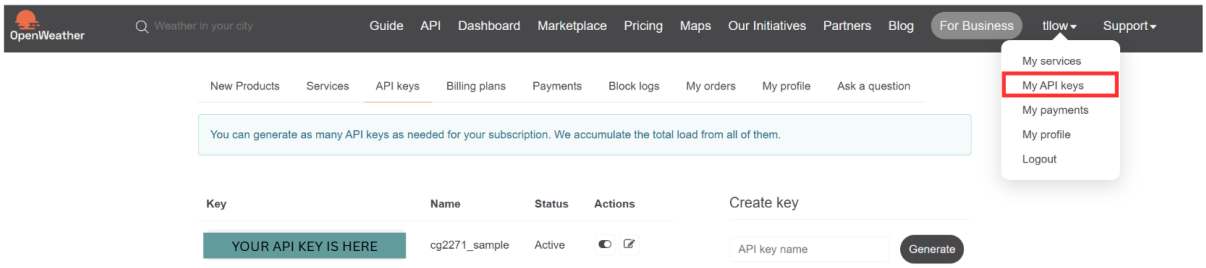
Annex B: Making an API Call

Once you have connected to the Internet, you are able to send and receive data from online sources. One such example is making an API call.

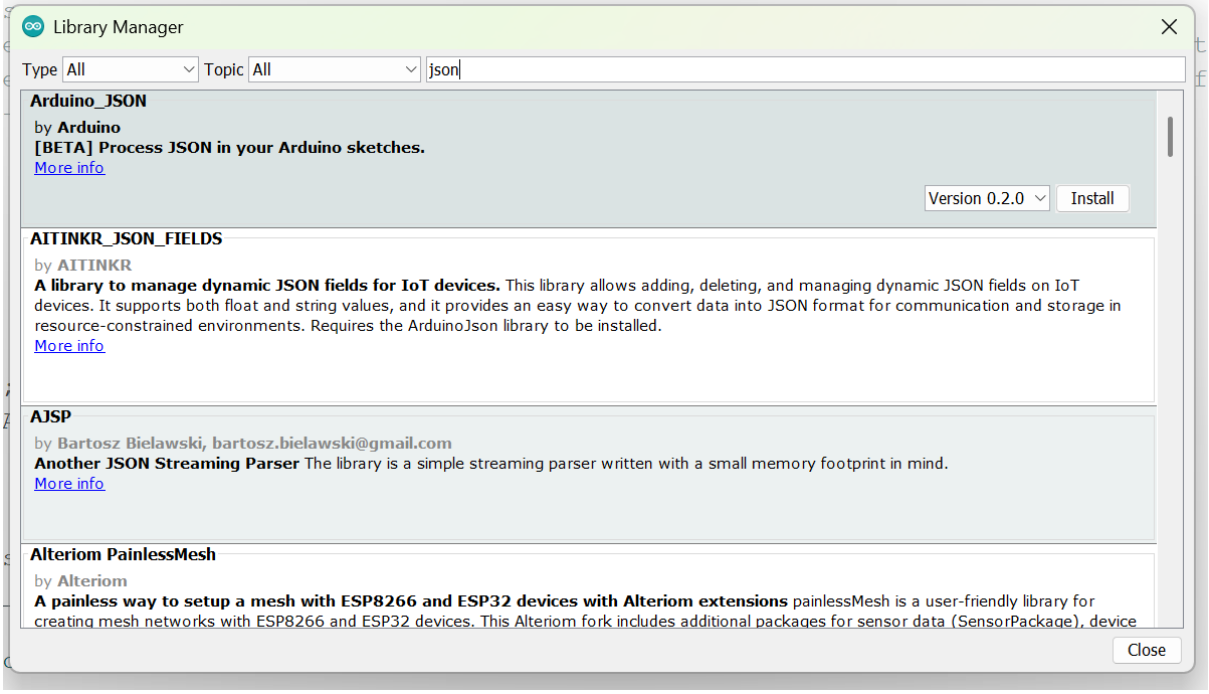
Once you connect to the Internet, you are able to send and receive data from other online sources. In this minimal example, we will be getting weather data from openweathermap.org. There are many tiers, but for this example, the free tier will do. Head to openweathermap.org and sign up for a new account.



Next, you will be asked to verify your account, and once you do so, you will be given an API key.



In the same example sketch, we use the library `HttpClient.h` to send and receive data. The library `Arduino_JSON.h` is used to parse the received data from the API. Note that different API calls will return different values and data types, so please be careful when using different API. If you run into compilation issues, you may need to install the Arduino_JSON library. From the menu in the top left corner, under Sketch > Include Library > Manage Libraries, search for "JSON" and install it.



In `as4_sample.ino` given, we connect to Wi-Fi, before sending a HTTP GET request to the API, to get a JSON output. Replace the `ssid`, `password` and `openWeatherMapApiKey` with your own values, and upload it to your ESP32. You should be able to see something like this in your serial monitor:



Other examples of using online functions include creating and controlling a Web App, polling and updating an online database, or communicating with Generative AI models.